COVID-19 / Public Health Data Visualization Dashboard

A Project Report Submitted in

the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

G Suvin Sai– 2420030115

D. Sriram– 2420030192

K. Vamsidhar– 2420030341

Under the Esteemed Guidance of

B Prathusha
Associate Professor

Department of Computer Science and Engineering

AY-2026–2027

Semester: Odd

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



Declaration

The Project Report entitled "**COVID-19 / Public Health Data Visualization Dashboard**" is a record of Bonafide work of **G Suvin Sai– 2420030115, D. Sriram– 2420030192, K. Vamsidhar– 2420030341**submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

G Suvin Sai– 2420030115

D. Sriram– 2420030192

K. Vamsidhar– 2420030341

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled "**COVID-19 / Public Health Data Visualization Dashboard**" is a bonafide work done and submitted by **G Suvin Sai– 2420030115, D. Sriram– 2420030192, K. Vamsidhar– 2420030341** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

_____	_____	_____
Signature of the Guide	Course Coordinator	Head of the Department
B Prathusha	Dr. N. Sunanda	Dr. P. Venkateswara Rao
Associate Professor, CSE	Department of CSE	Department of CSE

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator and my guide **Dr. N. Sunanda Associate Professor,** Department of Computer Science and Engineering which cannot be expressed in words for her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P. Venkateswar Rao**, Head of the ***Department of Computer Science and Engineering*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

G Suvin Sai– 2420030115

D. Sriram– 2420030192

K. Vamsidhar– 2420030341

ABSTRACT

The **COVID-19 / Public Health Data Visualization Dashboard** is a data science-based analytical system designed to analyze and visualize COVID-19 and public health data to identify important trends, patterns, and relationships. The project uses a comprehensive COVID-19 dataset containing attributes such as confirmed cases, active cases, recovered cases, deaths, vaccination rates, testing data, and geographical information. The system applies data preprocessing and exploratory data analysis to handle missing values, remove inconsistencies, understand data distributions, and identify relationships between different public health indicators. The analysis examines the spread and progression of COVID-19 across different regions and time periods, providing insights into infection rates, mortality, recovery, and vaccination trends.

For data analysis and visualization, the system applies statistical analysis and data visualization techniques to transform raw public health data into meaningful information. Various charts, graphs, time-series plots, and geographical visualizations are used to compare COVID-19 cases, deaths, recoveries, and vaccination rates across regions. The dashboard allows users to interact with the data by applying filters based on location and time period, making it easier to explore specific trends and compare public health conditions. The visualizations provide a clear representation of the progression of the pandemic and help identify regions and periods with significant changes in COVID-19 activity.

The selected analytical results are integrated into an interactive dashboard where users can explore key public health indicators and obtain meaningful insights from the dataset. The dashboard also presents statistical summaries, trend analysis, regional comparisons, and visual analytics to improve data interpretation. The primary objective is to demonstrate how **data science, exploratory data analysis, and data visualization** can be used to understand large-scale public health datasets and support data-driven decision-making. The system is intended as an analytical and visualization tool for understanding COVID-19 trends and public health patterns and does not replace official medical, epidemiological, or public health guidance.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	4
		Abstract	5
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Importance of the Application 1.6 Target Users/Audience	
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	
3	Technology Stack	3.1 Front-end – Streamlit 3.2 Application Logic – Python 3.3 Machine Learning – Scikit-learn 3.4 Image Processing – Pillow and NumPy 3.5 Data Visualization – Matplotlib 3.6 Model Storage – Joblib	
4	System Architecture	4.1 High-Level Architecture Diagram 4.2 Description of System Modules 4.3 Application Execution Architecture	
5	Design	5.1 User Interface Design and Screenshots 5.2 System Workflow / Data Flow Diagram	
6	Implementation	6.1 Module-wise Implementation 6.2 User Input and Image Upload 6.3 Image Preprocessing 6.4 KNN Model Implementation 6.5 SVM Model Implementation 6.6 Prediction and Performance Analysis	
7	Features	7.1 List of Main Features 7.2 Working of Each Feature 7.3 Interactive Drawing and Image Upload 7.4 Model Comparison Dashboard	
8	Testing	8.1 Functional Testing 8.2 Integration Testing 8.3 Tools Used for Testing 8.4 Test Cases and Results 8.5 Model Performance Evaluation	
9	Deployment	9.1 Steps to Run the Application 9.2 Python Environment Configuration 9.3 Local Application Execution	
10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations	
11	Future Enhancements	11.1 Planned Features 11.2 Deep Learning Integration 11.3 Character Recognition and Cloud Deployment	

12	Conclusion	12.1 Summary of the Project 12.2 Achievements 12.3 Skills Learned During Development	
13	References	Research Papers, Dataset and Technical Documentation	
14	Appendices	Application Screenshots Sample Code Snippets Installation and Setup Instructions User Manual	

CHAPTER 1

Introduction

1.1 Background of the Project

The COVID-19 pandemic has created significant challenges in monitoring and understanding public health conditions due to the continuous changes in confirmed cases, active cases, recoveries, deaths, and vaccination rates across different regions. The large volume of public health data makes it difficult to manually analyze, compare, and identify meaningful trends and patterns. Variations in infection rates and other health indicators over different time periods further increase the complexity of understanding the progression of the pandemic.

Data Science and data visualization techniques can help address this problem by analyzing public health datasets and presenting important information in an understandable format. In this project, we aim to develop a COVID-19 / Public Health Data Visualization Dashboard that collects and analyzes COVID-19-related data using data preprocessing, exploratory data analysis, and statistical techniques. The system provides interactive visualizations such as charts,graphs, and time-series plots to examine confirmed cases, active cases,location and time period and compare public health conditions across different regions.

1.2 Problem Statement

Analyzing COVID-19 data effectively is challenging because public health conditions change across different regions and time periods. Variations in confirmed cases, active cases, recoveries, deaths, testing data and vaccination rates make it difficult to understand the overall progression of the pandemic.

Traditional manual analysis of large public health datasets requires significant time and effort. Identifying trends, comparing regional statistics and

understanding relationships between different health indicators can also become difficult without proper visualization techniques.

Therefore, the main problem addressed in this project is to develop an interactive COVID-19 / Public Health Data Visualization Dashboard that can analyze public health datasets, preprocess the data, identify meaningful patterns and present the results through clear and informative visualizations.

The project also aims to provide statistical summaries, trend analysis and regional comparisons using charts, graphs and time-series visualizations,

allowing users to explore COVID-19 trends through location-based and time-period filters and gain meaningful insights into public health conditions.

1.3 Scope of the Project

The scope of this project includes:

1. Collecting COVID-19 and public health datasets containing relevant health indicators.
2. Analyzing confirmed COVID-19 cases across different regions.
3. Examining active cases, recovered cases and death statistics.
4. Analyzing vaccination rates and testing data.
5. Performing data preprocessing to handle missing values and inconsistencies.
6. Applying Exploratory Data Analysis (EDA) to identify meaningful patterns and trends.
7. Analyzing the progression of COVID-19 across different time periods.
8. Comparing public health statistics across different geographical regions.
9. Creating interactive visualizations using charts, graphs and time-series plots.
10. Providing filters based on geographical location and time period.
11. Displaying statistical summaries and important public health indicators.
12. Identifying relationships between different public health indicators.
13. Presenting analytical results through an interactive visualization dashboard.

14. Supporting data-driven understanding of COVID-19 trends and public health conditions.

The current implementation focuses on COVID-19 data analysis and visualization. It is intended as an analytical tool for understanding public

health trends and does not replace official medical, epidemiological or public health guidance.

1.4 Objectives

The main objectives of this project are:

1. To develop an interactive COVID-19 / Public Health Data Visualization Dashboard.

2. To collect and analyze COVID-19 and public health datasets.

3. To preprocess the collected data by handling missing values and inconsistencies.

4. To analyze confirmed cases, active cases, recoveries and deaths across different regions.

5. To examine vaccination rates and testing data to understand public health conditions.

6. To identify meaningful trends, patterns and relationships using Exploratory Data Analysis (EDA).

7. To compare COVID-19 statistics across different geographical regions and time periods.

8. To visualize analytical results using charts, graphs and time-series plots.

9. To provide interactive filters based on location and time period for better data exploration.

10. To display statistical summaries and important public health indicators through a user-friendly dashboard.

11. To present meaningful insights that help users understand the progression of the pandemic and support data-driven analysis.

1.5 Importance of the Application

The COVID-19 / Public Health Data Visualization Dashboard plays an important role in simplifying the analysis and understanding of public health information.

The application demonstrates how Data Science and data visualization techniques can be used to analyze large amounts of COVID-19 data and identify important trends and patterns. It also helps users understand the progression of the pandemic by presenting information about confirmed cases, active cases, recoveries, deaths and vaccination rates through interactive visualizations.

This project provides practical knowledge of data preprocessing, Exploratory Data Analysis (EDA), statistical analysis, data visualization and dashboard development. It also demonstrates how raw public health data can be transformed into meaningful insights to support better understanding and data-driven decision-making.

1.6 Target Users/Audience

The intended users of the application include:

1. Students learning Data Science, Data Analytics and data visualization.
2. Faculty members and academic researchers studying public health trends.
3. Public health researchers analyzing COVID-19 statistics.
4. Government and healthcare organizations interested in understanding pandemic trends.
5. Data analysts experimenting with Exploratory Data Analysis (EDA) and visualization techniques.
6. Users interested in exploring COVID-19 cases, recoveries, deaths and vaccination trends.
7. Educational institutions demonstrating data science and visualization applications.

CHAPTER 2

System Requirements

2.1 Hardware Requirements

The following hardware configuration is suitable for running the project.

```
Processor       : Intel Core i3 / i5 or equivalent
RAM             : 4 GB minimum, 8 GB recommended
Storage         : At least 1 GB of free space
Operating Input : Keyboard and Mouse
Display         : Standard Laptop or Desktop Display
GPU             : Optional
```

2.2 Software Requirements

The software requirements used for the project are:

1. Operating System: Windows 10 or Windows 11.
2. Programming Language: Python 3.x.
3. Development Environment: Visual Studio Code / Jupyter Notebook.
4. Application Framework: Streamlit.
5. Machine Learning Library: Scikit-learn.
6. Numerical Processing: NumPy.
7. Data Handling: Pandas, where used.
8. Data Visualization: Matplotlib.
9. Image Processing: Pillow.
10. Model Storage: Joblib.

2.3 Development Tools and Frameworks

Python:

Python is the main programming language used in this project. It is used for image processing, model training, prediction and application development.

Streamlit:

Streamlit is used to create the interactive web application. It provides the drawing canvas, image upload interface, prediction results and performance dashboard.

Scikit-learn:

Scikit-learn provides the Machine Learning algorithms used in the project. It is used to implement KNN and SVM and evaluate their performance.

NumPy:

NumPy is used for numerical calculations and handling image data in the form of arrays.

Pandas:

Pandas is useful for handling structured data and organizing evaluation results wherever required in the implementation.

Matplotlib:

Matplotlib is used to generate graphs and visual representations of model performance.

Pillow:

Pillow is used to open, convert, resize and process image files.

Joblib:

Joblib is used to save trained Machine Learning models and load them when the application runs.

CHAPTER 3

Technology Stack

3.1 Front-end – Streamlit

Streamlit is an open-source Python framework used to build interactive web applications.

In this project, Streamlit is used to create the user interface. It allows users to draw handwritten digits, upload images, submit inputs and view prediction results.

It also displays the model comparison dashboard with graphs, performance metrics and confusion matrices.

3.2 Application Logic – Python

Python is used to develop the main application logic. It manages image preprocessing, model loading, prediction generation and performance analysis.

Python also connects the different project modules and ensures that the input image moves through the complete recognition process.

3.3 Machine Learning – Scikit-learn

Scikit-learn is a Machine Learning library used for implementing classification algorithms.

In this project, two algorithms are implemented:

K-Nearest Neighbors (KNN): A distance-based classification algorithm that predicts a class based on the nearest training samples.

Support Vector Machine (SVM): A classification algorithm that learns decision boundaries to distinguish between different digit classes.

3.4 Image Processing – Pillow and NumPy

Pillow is used to read and manipulate images, while NumPy is used to represent image data as numerical arrays.

These tools support grayscale conversion, image resizing, pixel manipulation and normalization.

3.5 Data Visualization – Matplotlib

Matplotlib is used to generate visualizations for model evaluation and comparison.

It helps display performance graphs, confusion matrices and other analysis results.

3.6 Model Storage – Joblib

Joblib is used to store trained Machine Learning models in serialized files.

The application loads these saved models when it needs to generate predictions, avoiding the need to train the models every time the application starts.

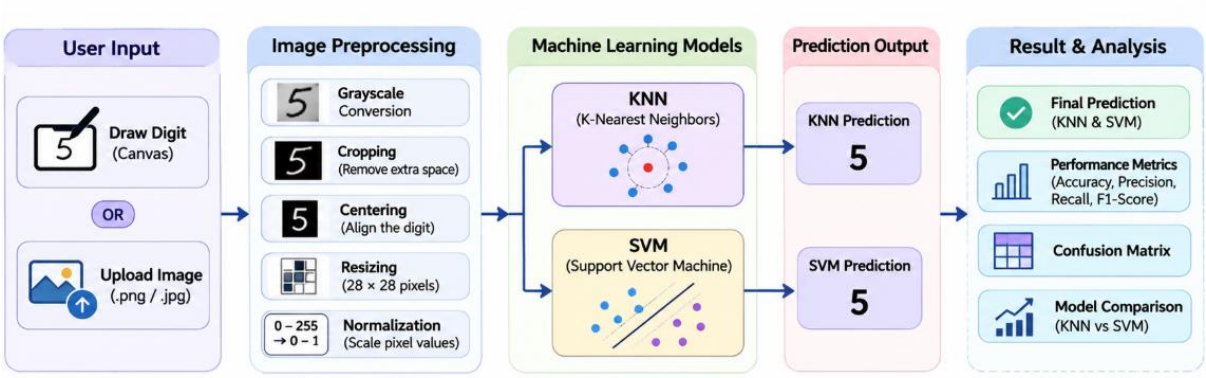
CHAPTER 4

System Architecture

4.1 High-Level Architecture Diagram

Insert the system architecture diagram here.

The architecture follows this sequence:



4.2 Description of System Modules

1. User Input Layer

This layer allows the user to provide a handwritten digit through the drawing canvas or image upload option.

2. Image Preprocessing Layer

The input image is prepared through grayscale conversion, cropping, centering, resizing and normalization.

3. Machine Learning Layer

The processed image is passed to the trained KNN and SVM models for classification.

4. Prediction Layer

The predictions from both models are generated and displayed to the user.

5. Performance Analysis Layer

This layer provides model evaluation results through accuracy, precision, recall, F1-score, graphs and confusion matrices.

4.3 Application Execution Architecture

The application is designed to run locally using Python and Streamlit.

The user starts the application from the project directory. Streamlit creates a local web interface that can be accessed through a browser.

The application uses the trained model files and required Python libraries to process images and generate predictions.

CHAPTER 5

Design

5.1 User Interface Design and Screenshots

The application is designed with a simple and interactive interface.

The main interface contains the following sections:

1. Main Application Screen

Displays the project interface and available input options.

2. Interactive Drawing Canvas

Allows users to draw handwritten digits directly using a mouse or supported input device.

3. Image Upload Section

Allows users to select and upload a handwritten digit image from their computer.

4. Preprocessed Image Display

Displays the processed 28 × 28 image before classification.

5. Prediction Results

Displays the predictions generated by KNN and SVM.

6. Performance Dashboard

Displays evaluation metrics, comparison graphs, confusion matrices and error analysis.

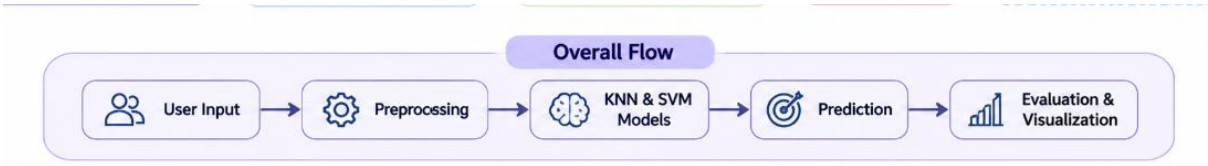
Insert the screenshots of the actual working application under this section.

5.2 System Workflow / Data Flow Diagram

A Data Flow Diagram represents how data moves through the system.

In this project, the input image is provided by the user and passed to the preprocessing module. After preprocessing, the image is given to the KNN and SVM models. Both models generate predictions, which are displayed through the application interface.

The overall data flow is:



CHAPTER 6

Implementation

6.1 Module-wise Implementation

The project is divided into four major modules.

Module 1: User Input Module

This module allows users to provide handwritten digit images through an interactive drawing canvas or by uploading an image.

Input: Handwritten digit image.

Output: Input image ready for preprocessing.

Module 2: Image Preprocessing Module

This module prepares the input image for classification. It performs grayscale conversion, inversion when required, cropping, centering, resizing and normalization.

Input: Raw handwritten image.

Output: Processed 28 × 28 image.

Module 3: Machine Learning Classification Module

This module uses KNN and SVM algorithms to recognize handwritten digits. Both trained models independently generate predictions.

Input: Processed image.

Output: Predicted digit from KNN and SVM.

Module 4: Prediction and Performance Analysis Module

This module displays the prediction results and compares the performance of both algorithms using evaluation metrics and visualizations.

Input: Model predictions and evaluation results.

Output: Predicted digit, accuracy, graphs, precision, recall, F1-score and confusion matrices.

6.2 User Input and Image Upload

The user input module provides two ways to submit handwritten digits.

The first option is an interactive drawing canvas where the user draws a digit directly.

The second option allows the user to upload an image containing a handwritten digit.

After receiving the input, the application sends the image to the preprocessing module.

6.3 Image Preprocessing

Image preprocessing is an important step because Machine Learning models require input images in a consistent format.

The following operations are performed:

Grayscale Conversion:

The input image is converted into grayscale to reduce unnecessary color information.

Image Inversion:

The image is inverted when required to match the expected background and digit color format.

Cropping:

Unnecessary empty areas around the handwritten digit are removed.

Centering:

The digit is positioned properly within the image.

Resizing:

The image is resized to 28 × 28 pixels to match the MNIST dataset format.

Normalization:

Pixel values are converted from the range 0–255 into the range 0–1.

After preprocessing, the image is represented using 784 pixel values.

6.4 KNN Model Implementation

K-Nearest Neighbors is a supervised Machine Learning classification algorithm.

It predicts the class of a new input by identifying the nearest training samples.

In this project, KNN is implemented with K = 3 and Euclidean distance.

The algorithm follows these steps:

1. Load the training dataset.
2. Calculate the distance between the input and training samples.
3. Identify the three nearest neighbors.
4. Apply majority voting.
5. Predict the digit class.

The KNN model is used as a distance-based classification approach for recognizing handwritten digits.

6.5 SVM Model Implementation

Support Vector Machine is a supervised Machine Learning algorithm used for classification.

SVM learns decision boundaries that help distinguish different digit classes.

In this project, the RBF kernel is used with C = 10.

The algorithm follows these steps:

1. Load the training dataset.
2. Train the SVM classifier.
3. Learn decision boundaries between digit classes.
4. Process the input image.
5. Predict the digit class.

The RBF kernel helps the model classify complex patterns that cannot always be separated using simple linear boundaries.

6.6 Prediction and Performance Analysis

After preprocessing, the same input image is passed to both KNN and SVM.

Each model independently generates a prediction.

The application displays both outputs so that users can compare the predictions.

The performance dashboard evaluates both models using:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrices
- Model comparison graphs
- Class-wise performance
- Misclassified image analysis
- Robustness analysis

CHAPTER 7

Features

7.1 List of Main Features

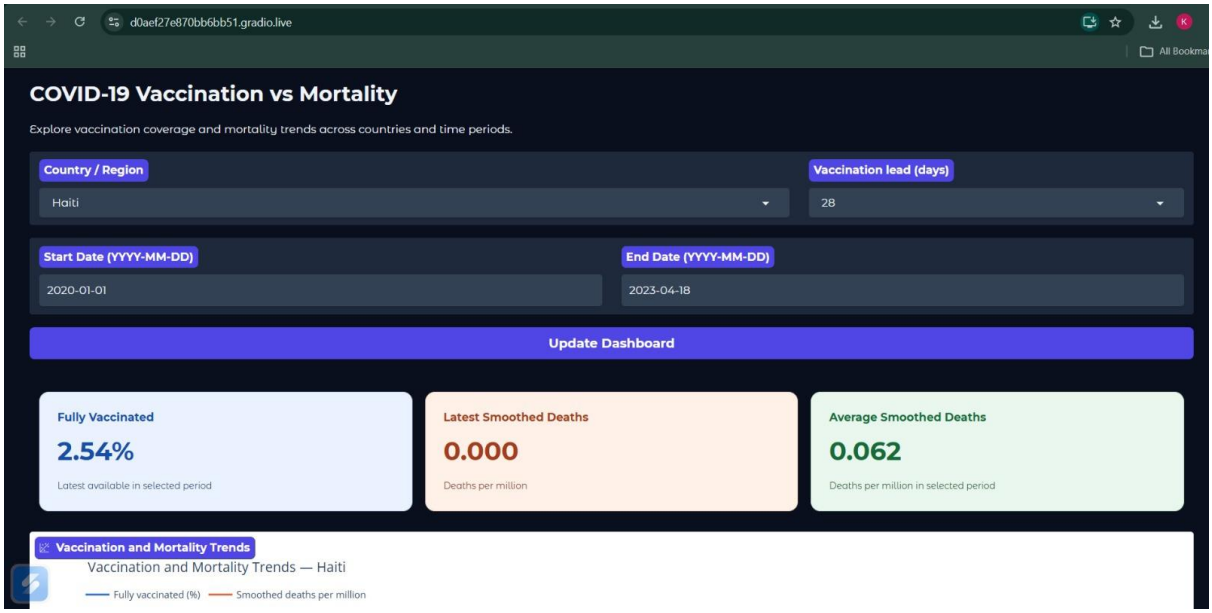
The major features of the application are:

1. Interactive digit drawing.
2. Handwritten image upload.
3. Automatic image preprocessing.
4. KNN-based digit recognition.
5. SVM-based digit recognition.
6. Side-by-side prediction display.
7. Model agreement and disagreement display.
8. Performance comparison dashboard.
9. Confusion matrix visualization.
10. Misclassified image analysis.
11. Robustness analysis using image variations.

7.2 Working of Each Feature

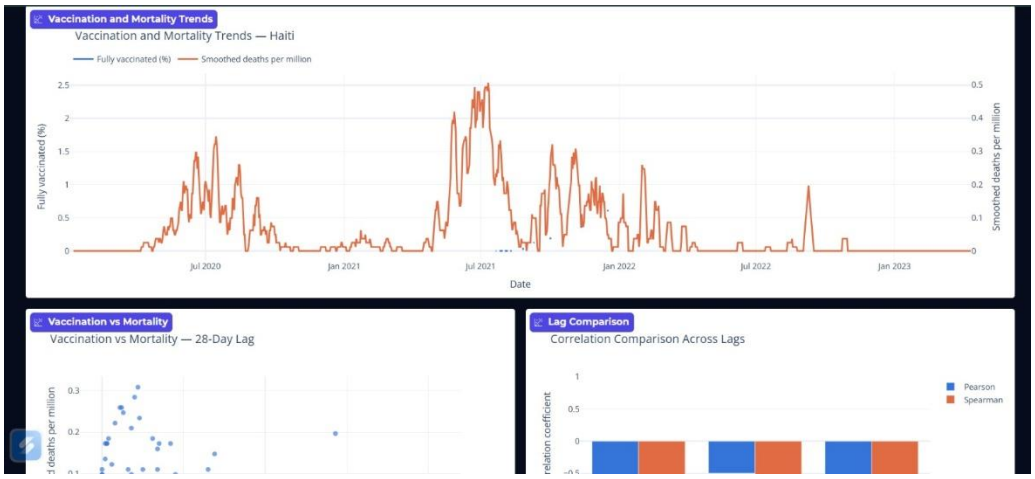
Country/Region Selection:

The user selects a particular country or region from the dropdown menu. The dashboard displays COVID-19 vaccination and mortality information based on the selected location.



Date Range Selection:

The user selects the start date and end date to analyze COVID-19 data for a specific time period.



Update Dashboard:

After selecting the required filters and date range, the user clicks the Update Dashboard button. The dashboard updates the visualizations and statistical information based on the selected inputs.

Key Performance Indicators (KPIs):

The dashboard displays important statistical values such as public vaccination percentage, latest smoothed deaths and average smoothed deaths. These indicators provide a quick overview of public health conditions.

Vaccination and Mortality Trends:

The dashboard uses a time-series line chart to display vaccination and mortality trends over different time periods. It helps users understand how these indicators change over time.

Vaccination vs Mortality Analysis:

A scatter plot is used to examine the relationship between vaccination rates and mortality rates. It helps identify patterns and possible relationships between the two indicators.

Relationship Analysis:

The dashboard displays statistical values such as correlation and lag correlation to understand the relationship between vaccination and mortality data.

7.3 Interactive Dashboard and Data Filtering

The application provides an interactive dashboard where users can explore COVID-19 vaccination and mortality data.

Users can select a country or region and specify a date range to view the required information. After applying the filters, the dashboard updates the KPI values, charts and statistical results.

7.4 Vaccination and Mortality Visualization

The dashboard provides different visualizations to understand vaccination and mortality patterns.

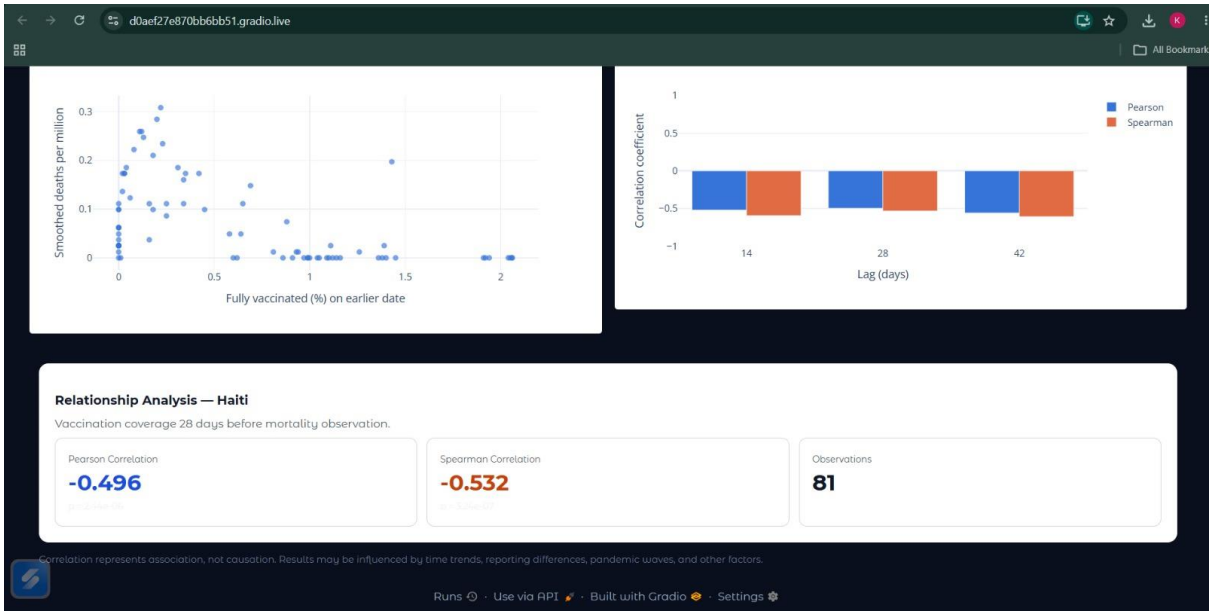
The visualizations include:

1. Time-Series Line Chart: Displays vaccination and mortality trends over time.

2. Scatter Plot: Shows the relationship between vaccination rates and mortality rates.

3. Lag Comparison Chart: Compares vaccination and mortality values across different time lags.

4. Statistical Analysis: Displays correlation and lag correlation values to identify relationships between the indicators.



CHAPTER 8

Testing

8.1 Functional Testing

Functional testing is performed to verify whether the main features of the application work as expected.

The following operations are tested:

1. Selecting a country or region from the dropdown menu.
2. Selecting the start date and end date.
3. Updating the dashboard based on the selected filters.
4. Displaying public vaccination percentage.
5. Displaying latest smoothed deaths.
6. Displaying average smoothed deaths.
7. Generating vaccination and mortality trend charts.
8. Displaying vaccination and mortality scatter plots.
9. Performing lag comparison analysis.
10. Displaying correlation and lag correlation values.
11. Updating visualizations based on the selected location and time period.
12. Displaying statistical summaries and analytical results.

8.2 Integration Testing

Integration testing checks whether the different modules work together correctly.

The following sequence is verified:

1. The user selects a country or region and specifies the start date and end date.
2. The dashboard receives the selected inputs.
3. The system retrieves the corresponding COVID-19 vaccination and mortality data.
4. The data is processed and analyzed to identify trends and relationships.

5. The system generates vaccination and mortality visualizations.
6. Statistical results such as correlation and lag correlation are calculated.
7. The dashboard displays the KPI values, charts and performance information to the user.

8.3 Tools Used for Testing

The following tools and libraries are used:

- Python for executing the application.
- Streamlit for testing the user interface.
- Scikit-learn for model evaluation.
- NumPy for numerical operations.
- Matplotlib for visualizing results.
- VS Code and Jupyter Notebook for development and testing.

Postman testing is not applicable because the current implementation does not use a separate REST API backend.

8.4 Test Cases and Results

Test Case ID	Test Case	Expected Result	Status
TC-01	Select a country or region	Selected location should be accepted	Pass (if successfully tested)
TC-02	Select start date and end date	Selected date range should be applied	Pass (if successfully tested)
TC-03	Click Update Dashboard	Dashboard should update according to selected filters	Pass (if successfully tested)
TC-04	Display KPI values	Vaccination percentage and mortality statistics should be displayed	Pass (if successfully tested)
TC-05	Open performance dashboard	Evaluation metrics and graphs should be displayed	Pass (if successfully tested)

8.5 Model Performance Evaluation

The models are evaluated using the MNIST test dataset.

The following results were obtained in the latest evaluation discussed for this project.

Evaluation Metric	KNN	SVM
Accuracy	97.05%	97.67%
Precision	97.09%	97.68%
Recall	97.01%	97.65%
F1-score	97.04%	97.66%

The results indicate that SVM achieved higher performance than KNN across the reported evaluation metrics.

SVM achieved 97.67% accuracy, while KNN achieved 97.05% accuracy.

Therefore, SVM performed slightly better on the evaluated MNIST test dataset.

CHAPTER 9

Deployment

9.1 Steps to Run the Application

The application can be executed locally using the following steps:

1. Install Python on the computer.
2. Open the project folder in Visual Studio Code.
3. Create a Python virtual environment.
4. Activate the virtual environment.
5. Install the required Python libraries.
6. Ensure that the trained KNN and SVM model files are available.
7. Run the Streamlit application.

Command to execute the application:

```
python -m streamlit run app.py
```

After executing the command, Streamlit displays a local URL in the terminal.

Open that URL in a web browser to access the application.

9.2 Python Environment Configuration

The project uses a Python virtual environment to manage the required libraries.

The environment contains the necessary packages for image processing, Machine Learning, data handling and application development.

The trained model files must be available in the correct project directory so that the application can load them successfully.

9.3 Local Application Execution

The application runs locally through Streamlit.

The user interface and application logic operate within the same Python application environment.

Separate front-end and back-end hosting are not required for the current implementation.

CHAPTER 10

Challenges and Limitations

10.1 Issues Faced During Development

During the development of this project, we faced several challenges.

Missing and Inconsistent Data:

The COVID-19 dataset contained missing values and inconsistencies in certain records, which affected the analysis.

Different Country Data:

COVID-19 vaccination and mortality data varied across different countries and regions, making comparison difficult.

Date Range Selection:

Selecting specific start and end dates and displaying the corresponding data required proper filtering.

Data Visualization:

Presenting large amounts of COVID-19 data through charts and graphs in an understandable format was challenging..

Dashboard Integration: Integrating KPI cards, charts, filters and statistical results into a single interactive dashboard required careful development.

10.2 Solutions Applied

The following solutions were applied:

1. Data preprocessing was used to handle missing values and inconsistencies.
2. Data filtering was applied to select specific countries and regions.
3. Date-based filtering was implemented to analyze data for selected time periods.
4. Statistical analysis was used to identify trends and relationships.
5. Time-series charts were used to visualize vaccination and mortality trends.
6. Scatter plots were used to examine the relationship between vaccination and mortality.
7. Lag comparison was implemented to analyze relationships across different time intervals.
8. KPI cards were used to display important statistical information.
9. An interactive dashboard was developed to integrate all visualizations and analytical results.

10.3 Current Limitations

The current system has the following limitations:

1. The application recognizes numerical digits from 0 to 9 only.
2. The models are trained primarily using the MNIST dataset.
3. Unusual handwriting styles may reduce prediction accuracy.
4. Poorly written or ambiguous digits may be misclassified.
5. The system does not currently recognize alphabets or complete sentences.
6. The current implementation is designed for local execution.

CHAPTER 11

Future Enhancements

11.1 Planned Features

The project can be extended in several ways.

Deep Learning Integration:

Convolutional Neural Networks (CNNs) can be introduced to improve handwritten digit recognition.

Character Recognition:

The system can be extended to recognize handwritten alphabets and special characters.

Real-Time Recognition:

Continuous handwritten input and real-time prediction can be introduced.

Mobile Application:

A mobile-friendly application can be developed to make the system accessible through smartphones.

Cloud Deployment:

The application can be hosted online so that users can access it from different devices.

Dataset Expansion:

Additional handwritten samples from different users can be included to improve generalization.

11.2 Deep Learning Integration

In future development, CNN-based models can be implemented to automatically learn important image features.

These models may improve recognition performance by learning complex patterns from handwritten images.

11.3 Character Recognition and Cloud Deployment

The application can be extended to recognize alphabets and complete handwritten text.

Cloud deployment can also be introduced to make the application accessible online without requiring local installation.

CHAPTER 12

Conclusion

12.1 Summary of the Project

The project **"COVID-19 / Public Health Data Visualization Dashboard"** was developed as an interactive data analysis and visualization application.

The system allows users to select a country or region and choose a specific date range to explore COVID-19 vaccination and mortality data. The selected data is processed and analyzed using data preprocessing and Exploratory Data Analysis (EDA) techniques.

The application displays important statistical information through KPI cards, including public vaccination percentage, latest smoothed deaths and average smoothed deaths. It also provides visualizations such as time-series charts, scatter plots and lag comparison charts to understand vaccination and mortality trends.

The dashboard provides relationship analysis using correlation and lag correlation values to examine the relationship between vaccination rates and mortality. Users can interact with the dashboard by changing location and date filters to explore different patterns and trends.

The project demonstrates how Data Science, statistical analysis and data visualization techniques can be combined to transform raw public health data

into meaningful insights and support better understanding of COVID-19 trends.

12.2 What Was Achieved

The major achievements of the project include:

1. Developed an interactive COVID-19 / Public Health Data Visualization Dashboard.
2. Implemented country and region selection functionality.
3. Provided date range selection for analyzing specific time periods.
4. Applied data preprocessing techniques to handle missing values and inconsistencies.
5. Analyzed COVID-19 vaccination and mortality data.
6. Displayed important public health indicators using KPI cards.
7. Visualized vaccination and mortality trends using time-series charts.
8. Implemented scatter plots to examine vaccination and mortality relationships.
9. Developed lag comparison visualizations to analyze changes across different time intervals.
10. Performed correlation and lag correlation analysis.
11. Developed an interactive dashboard to display statistical summaries and visualizations.
12. Enabled users to explore public health trends through interactive filters.

12.3 Skills Learned During Development

Through this project, we gained practical experience in:

1. Python programming.
2. Image processing and preprocessing.
3. Machine Learning classification.
4. KNN and SVM algorithms.
5. Dataset handling.
6. Model training and evaluation.
7. Streamlit application development.
8. Data visualization.
9. Model saving and loading.
10. Team collaboration and technical documentation.

CHAPTER 13

References

1. Yann LeCun, Corinna Cortes and Christopher J. C. Burges, "The MNIST Database of Handwritten Digits." Available at: http://yann.lecun.com/exdb/mnist
2. Scikit-learn Documentation, "K-Nearest Neighbors Classification." Available at: https://scikit-learn.org/stable/modules/neighbors.html
3. Scikit-learn Documentation, "Support Vector Machines." Available at: https://scikit-learn.org/stable/modules/svm.html
4. Streamlit Documentation. Available at: https://docs.streamlit.io/
5. NumPy Documentation. Available at: https://numpy.org/doc/
6. Matplotlib Documentation. Available at: https://matplotlib.org/stable/
7. Pillow Documentation. Available at: https://pillow.readthedocs.io/
8. Joblib Documentation. Available at: https://joblib.readthedocs.io/
9. Chunmei Zhang et al., "ResNet18-ThunderSVM: Hybrid Intelligence for Handwritten Digit Recognition by Fusing Deep Spatial Features and High-Performance Classification," Scientific Reports, 2026.
10. Lei Guo and Zihan Wang, "Task-fMRI-Derived Number-Related Functional Brain Topology Constrained Spiking Neural Networks for Handwritten Digit Recognition," Applied Sciences, 2026.
11. Raghav Sethi and Varda Pareek, "Optical Handwritten Digit Recognition using Deep Learning Models on the MNIST Dataset," 2026.
12. Dan Zhao et al., "Handwritten Digit Recognition Based on Ghost Imaging and Cake-Cutting Hadamard Matrix," Physics Letters A, 2026.

CHAPTER 14

Appendices

14.1 Application Screenshots

The following screenshots demonstrate the working of the application:

1. Main application interface.
2. Interactive drawing canvas.
3. Handwritten image upload section.
4. Preprocessed image display.
5. KNN prediction output.
6. SVM prediction output.
7. Model comparison dashboard.
8. Confusion matrices.
9. Error analysis and robustness visualizations.

14.2 Sample Code Snippets

The following is a simplified example of the KNN and SVM implementation.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC

# Initialize KNN model
knn = KNeighborsClassifier(n_neighbors=3)

# Initialize SVM model
svm = SVC(kernel="rbf", C=10)

# Train both models
knn.fit(X_train, y_train)
svm.fit(X_train, y_train)

# Generate predictions
knn_prediction = knn.predict(X_test)
svm_prediction = svm.predict(X_test)
```

14.3 Installation and Setup Instructions

Step 1: Install Python 3.x.

Step 2: Open the project folder in Visual Studio Code.

Step 3: Activate the virtual environment.

```
.\venv\Scripts\Activate.ps1
```

Step 4: Install the required libraries.

```
pip install -r requirements.txt
```

Step 5: Run the application.

```
python -m streamlit run app.py
```

Step 6: Open the local URL displayed in the terminal.

14.4 User Manual

The application can be used through the following steps:

1. Open the Streamlit application in a browser.
2. Select the drawing canvas or image upload option.
3. Draw a handwritten digit or upload an image.
4. Click the Analyze and Predict button.
5. View the preprocessed image.
6. Check the predictions generated by KNN and SVM.
7. Compare the predictions from both models.
8. Open the performance dashboard.
9. View the evaluation metrics, graphs and confusion matrices.

14.5 Team Contributions

Suvin Sai:

- Developed the user input module.
- Worked on the interactive drawing canvas.
- Worked on image upload functionality.

- Worked on displaying KNN and SVM predictions.
- Worked on the model comparison and performance dashboard.
- Worked on result visualization and presentation.

D Sri Ram:

- Worked on the image preprocessing module.
- Implemented grayscale conversion.
- Worked on cropping and centering.
- Worked on image resizing.
- Worked on pixel normalization.

K. Vamsidhar:

- Worked on KNN implementation.
- Worked on SVM implementation.
- Worked on model training using the MNIST dataset.
- Worked on testing and prediction.
- Evaluated model performance using accuracy, precision, recall and F1-score.

All three team members contributed to project integration, testing, debugging and documentation according to their actual work.